PIMPRI CHINCHWAD EDUCATION TRUST's.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV SUD

T.Y. B. TECH **Year:** 2025 – 26 **Semester:** 6
Branch: Computer Engineering **Date:**13/4/2026

Assignment 7

Assignment Title: Design a web application for e-commerce or students feedback review systems or any other applications with MongoDB Backend. using React.js and Express.

Name: Trushita Sathe

PRN:124B2B024

DIV: C

Objectives:

- To develop a dynamic Student Feedback System using Node.js and Express
- To create a user-friendly interface for submitting feedback
- To integrate frontend with backend using Fetch API
- To store feedback data in MongoDB database

Outcomes:

- Feedback details stored in MongoDB database
- Automatic display of submitted feedback in sidebar
- Real-time UI update without page reload
- Ability to delete feedback entries

Theory Concept in brief:

The Student Feedback System is built using a full-stack JavaScript architecture. The system allows users to submit feedback with ratings and comments. It validates user inputs and dynamically updates the interface without reloading the page. The backend handles API requests while the frontend communicates using Fetch API to perform CRUD operations. MongoDB is used as a database to store feedback records.

Technology Stack Used

• HTML & CSS
Used to design and structure the user interface, including the booking form and sidebar display.

• JavaScript (Frontend)
Used to handle form submission, validations, dynamic updates, and API communication using Fetch API.

• Node.js
Server-side JavaScript runtime environment used to run the backend application.

• Express.js
Used to create REST APIs for handling appointment booking, retrieving appointments, and managing data.

• MongoDB
Used as NoSQL database to store appointment records.

Dataset used:

MongoDB

Used as database to store records.

Used as database to store feedback records.

Each record contains:

• student
• email
• rating
• comment
• createdAt

The database stores structured feedback information and allows retrieval and deletion of records dynamically.

Program code:

App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";

const API = "http://localhost:3001/api/feedback";

export default function App() {
  const [items, setItems] = useState([]);
  const [form, setForm] = useState({
    student: "",
    email: "",
    rating: 5,
    comment: "",
  });

  const ratingsText = ["Bad", "Okay", "Good", "Very Good", "Amazing"];

  async function load() {
    const res = await fetch(API);
    setItems(await res.json());
  }

  useEffect(() => {
    load();
  }, []);

  async function submit(e) {
    e.preventDefault();

    await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });

    setForm({ student: "", email: "", rating: 5, comment: "" });
    load();
  }

  async function del(id) {
    await fetch(`${API}/${id}`, { method: "DELETE" });
```

```
  load();
}

return (
  <div className="layout">

    {/* SIDEBAR (LEFT) */}
    <div className="sidebar">
      <h2>Feedback List</h2>

      {items.map((f) => (
        <div key={f._id} className="appointment-item">
          <b>{f.student}</b> ({f.rating}/5)
          <p>{f.comment}</p>
          <button onClick={() => del(f._id)}>Delete</button>
        </div>
      ))}
    </div>

    {/* MAIN CONTENT */}
    <div className="main-content">
      <div className="form-container">
        <h1>Student Feedback</h1>

        <form onSubmit={submit}>

          <label>Name</label>
          <input
            value={form.student}
            onChange={(e) =>
              setForm({ ...form, student: e.target.value })
            }
            required
          />

          <label>Email</label>
          <input
            value={form.email}
            onChange={(e) =>
              setForm({ ...form, email: e.target.value })
            }
          />

          <label>Rating</label>
          <div className="stars">
```

```
            {[1, 2, 3, 4, 5].map((n) => (
              <span
                key={n}
                className={form.rating >= n ? "star active" : "star"}
                onClick={() => setForm({ ...form, rating: n })}
              >
                ★
              </span>
            ))}
          </div>

          <div className="ratingText">
            {ratingsText[form.rating - 1]}
          </div>

          <label>Message</label>
          <textarea
            value={form.comment}
            onChange={(e) =>
              setForm({ ...form, comment: e.target.value })
            }
          />

          <button type="submit">Submit</button>

        </form>
      </div>
    </div>

  </div>
  );
}
```

App.css

```
/* LAYOUT */
.layout {
  display: flex;
  width: 100%;
  min-height: 100vh;
}

/* SIDEBAR */
```

```css
.sidebar {
  width: 320px;
  min-width: 320px;
  background: #1e3c72;
  color: white;
  padding: 20px;
  overflow-y: auto;
}

.sidebar h2 {
  margin-bottom: 20px;
}

/* FEEDBACK CARD */
.appointment-item {
  background: white;
  color: black;
  padding: 10px;
  border-radius: 6px;
  margin-bottom: 10px;
  font-size: 14px;
}

.appointment-item button {
  margin-top: 5px;
  padding: 5px;
  background: red;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

/* MAIN CONTENT */
.main-content {
  flex: 1;
  display: flex;
  justify-content: center;
  align-items: center;
}

/* FORM BOX */
.form-container {
  background: white;
  padding: 30px;
```

```
  width: 100%;
  max-width: 500px;
  border-radius: 8px;
  box-shadow: 0 5px 20px rgba(0,0,0,0.1);
}

.form-container h1 {
  margin-bottom: 20px;
  color: #1e3c72;
}

/* FORM */
label {
  font-weight: 600;
  font-size: 14px;
  margin-bottom: 5px;
  display: block;
}

input, textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

textarea {
  resize: none;
  height: 80px;
}

/* BUTTON */
button {
  width: 100%;
  padding: 10px;
  background: #1e3c72;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

button:hover {
  background: #16325c;
```

```
}

/* STARS */
.stars {
  text-align: center;
  margin-bottom: 10px;
}

.star {
  font-size: 26px;
  cursor: pointer;
  color: #ccc;
}

.star.active {
  color: gold;
}

/* RATING TEXT */
.ratingText {
  text-align: center;
  color: orange;
  font-weight: bold;
  margin-bottom: 10px;
}

/* RESPONSIVE */
@media(max-width: 900px) {
  .layout {
    flex-direction: column;
  }

  .sidebar {
    width: 100%;
  }
}
```

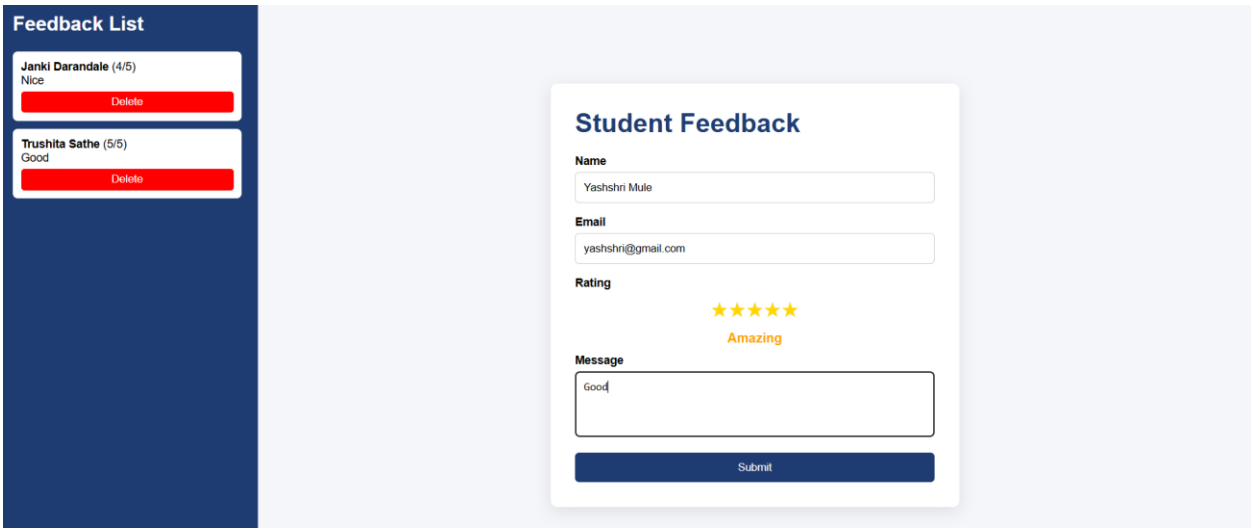
Server.js

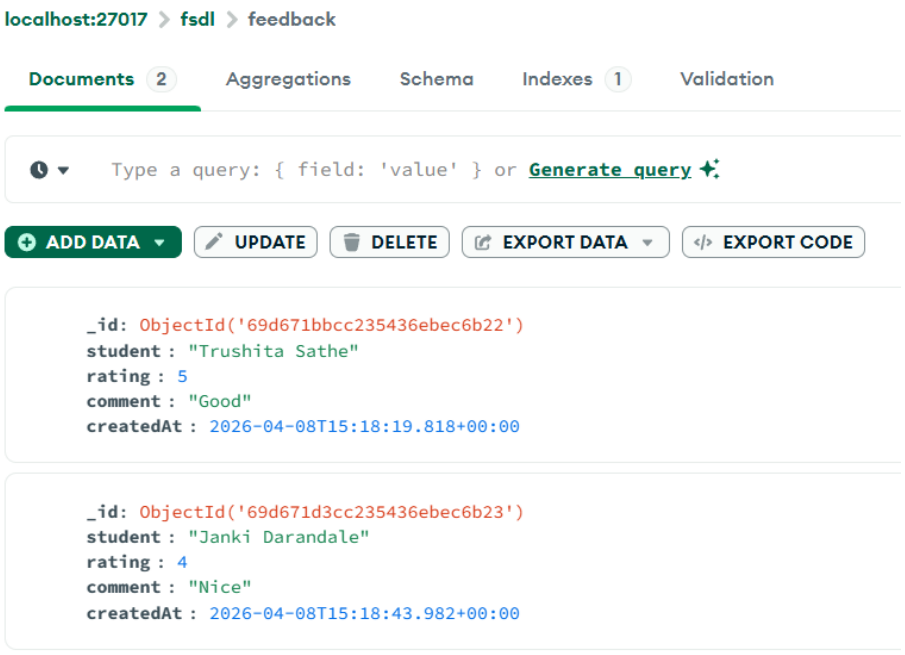
```
import express from "express";

import cors from "cors";
import { MongoClient, ObjectId } from "mongodb";
import dotenv from "dotenv";
```

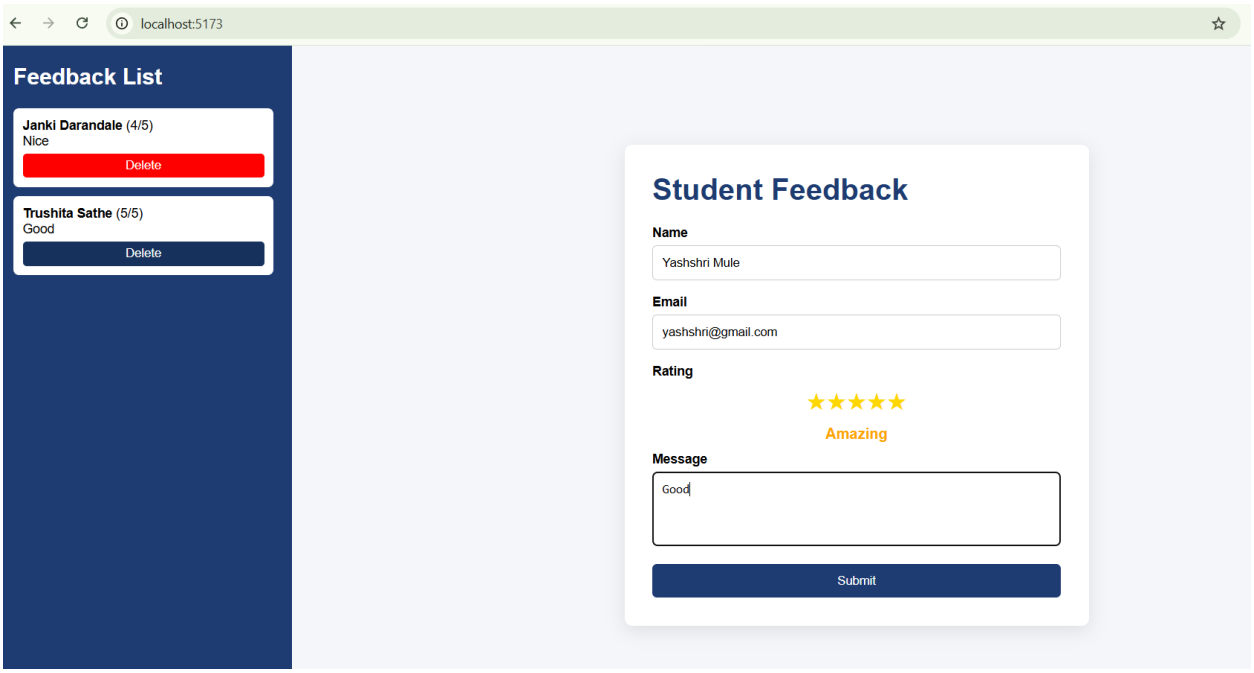
```
dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());

const client = new MongoClient(process.env.MONGO_URI);

async function start() {
  await client.connect();

  const db = client.db("fsdl");
  const feedback = db.collection("feedback");

  // GET
  app.get("/api/feedback", async (req, res) => {
    const data = await feedback.find().sort({ createdAt: -1 }).toArray();
    res.json(data);
  });

  // POST
  app.post("/api/feedback", async (req, res) => {
    const { student, email, rating, comment } = req.body;

    if (!student || !rating) {
      return res.status(400).json({ error: "Missing fields" });
    }

    await feedback.insertOne({
      student,
      email,
      rating: Number(rating),
      comment,
      createdAt: new Date(),
    });

    res.json({ ok: true });
  });

  // DELETE
  app.delete("/api/feedback/:id", async (req, res) => {
    await feedback.deleteOne({ _id: new ObjectId(req.params.id) });
    res.json({ ok: true });
  });
```

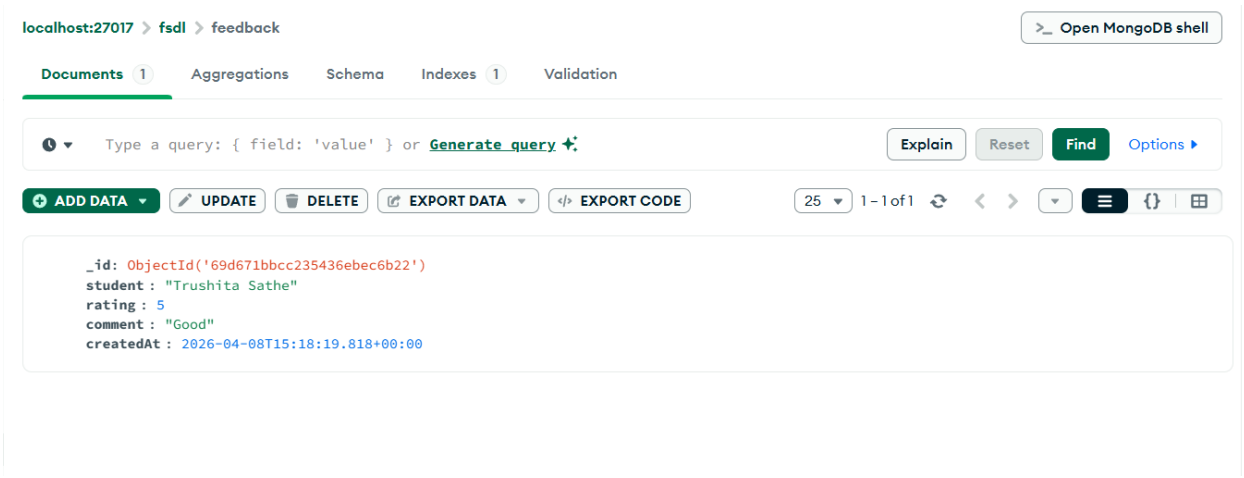
```
  app.listen(3001, () =>
    console.log("Server running on http://localhost:3001")
  );
}

start();
```

Output screenshot:









Conclusion:

The Student Feedback System successfully demonstrates the use of Node.js, Express, React, and MongoDB to build a dynamic full-stack web application. The system ensures proper validation, efficient data storage, and real-time UI updates. It provides a simple and effective platform for collecting and managing feedback digitally.